

Enlaces Colab:

https://colab.research.google.com/drive/17SisgRQ1c01zAvBu6DvDrSrMZL_yHEgV?usp=sharing

https://colab.research.google.com/drive/1RDw-dMh_carD2RmtxEDnoDwv9YO0DQ_K?usp=sharing

Implementación Manual vs. Keras

Perceptrón Multicapa Manual (3 capas ocultas):

Ventaja

Control total sobre el algoritmo, comprensión profunda de los mecanismos internos (propagación, retropropagación, actualización de pesos).

Desventaja

Más propenso a errores, requiere más código para funcionalidades básicas, optimización y escalabilidad limitadas.

Perceptrón Multicapa con Keras (3 capas ocultas):

Ventaja

Rápido de implementar, robusto, optimizado para rendimiento (utiliza backends como TensorFlow), fácil de escalar y experimentar con diferentes arquitecturas.

Desventajas

Menos control de bajo nivel, puede ocultar la complejidad interna del algoritmo.

2. Red Original (Keras Superficial) vs. Red Más Profunda (Keras Profundo y Manual)

Perceptrón Multicapa Keras Superficial (1 capa oculta):

Una red más simple puede ser suficiente para problemas linealmente separables o de baja complejidad, y puede ser menos propensa al sobreajuste si los datos son limitados.

Perceptrón Multicapa Keras Profundo (3 capas ocultas):

La adición de capas ocultas puede permitir a la red aprender patrones más complejos en los datos, lo que potencialmente conduce a una mayor precisión en problemas no lineales. Sin embargo, también puede aumentar el riesgo de sobreajuste y requiere más tiempo de entrenamiento y recursos computacionales.

Perceptrón Multicapa Manual Profundo (3 capas ocultas):

Similar al modelo Keras profundo en términos de capacidad de representación, pero su rendimiento real puede variar debido a diferencias en la inicialización, optimización y la implementación específica del algoritmo de retropropagación.

En general, para el desarrollo y la implementación rápida de modelos de aprendizaje automático, Keras es la opción preferida debido a su eficiencia, robustez y facilidad de uso. La implementación manual es útil para fines educativos y para comprender los principios fundamentales, pero no es práctica para aplicaciones a gran escala.

En cuanto a la profundidad de la red, para el conjunto de datos Iris, una red más profunda (3 capas ocultas) podría no mejorar significativamente la precisión en comparación con una red más superficial (1 capa oculta), o incluso podría requerir un ajuste más fino de hiperparámetros para evitar el sobreajuste. El rendimiento exacto dependerá de la complejidad del problema y de la cantidad y calidad de los datos de entrenamiento.

Ejecuciones

Keras:

[1 0 0]

[4]
✓ 0 s

```
model = keras.Sequential(  
    [  
        layers.Dense(3, activation="sigmoid", name="layer1", input_shape=(4,)),  
        layers.Dense(3, activation="sigmoid", name="layer2"),  
    ]  
)  
  
model.summary()
```

▼

... /usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input\_shape` argument to the `Dense` layer constructor. It has been deprecated since Keras 3.0.0. It will be removed in Keras 3.1.0. You can pass it to the `Dense` layer constructor for backwards compatibility, but it is not recommended.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"

Layer (type)	Output Shape	Param #
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12

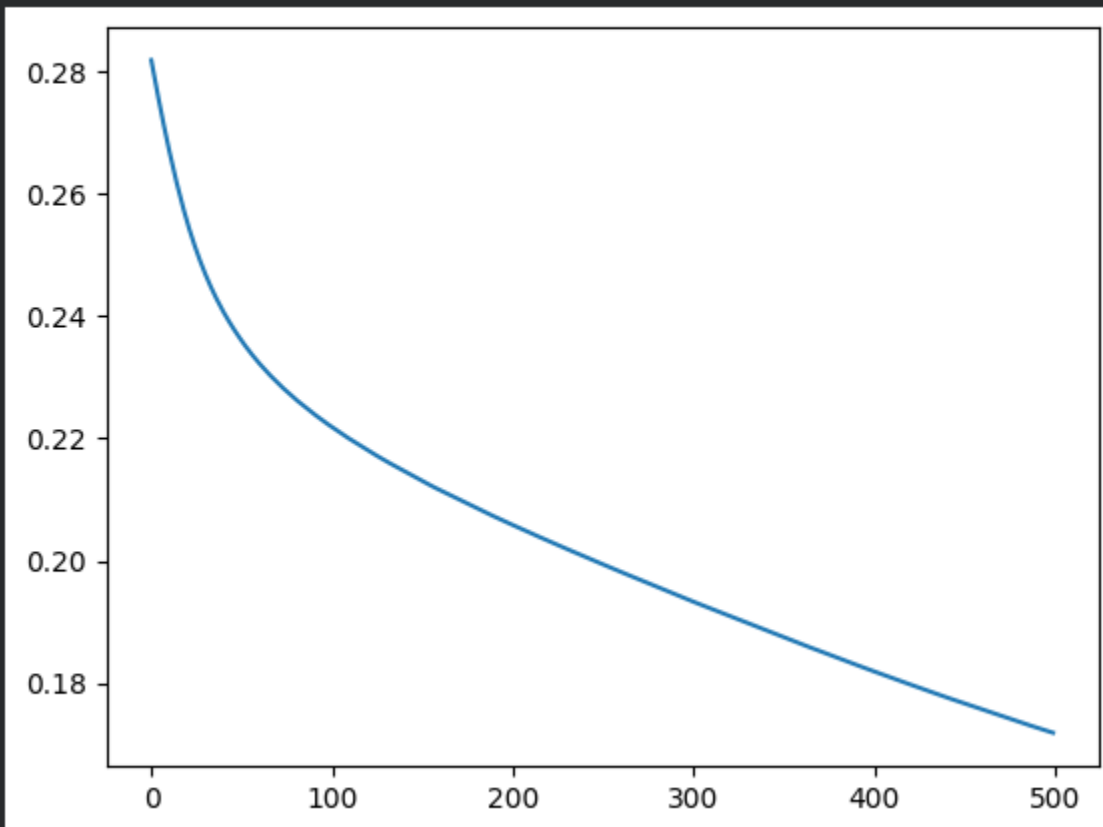
Total params: 27 (108.00 B)
Trainable params: 27 (108.00 B)
Non-trainable params: 0 (0.00 B)

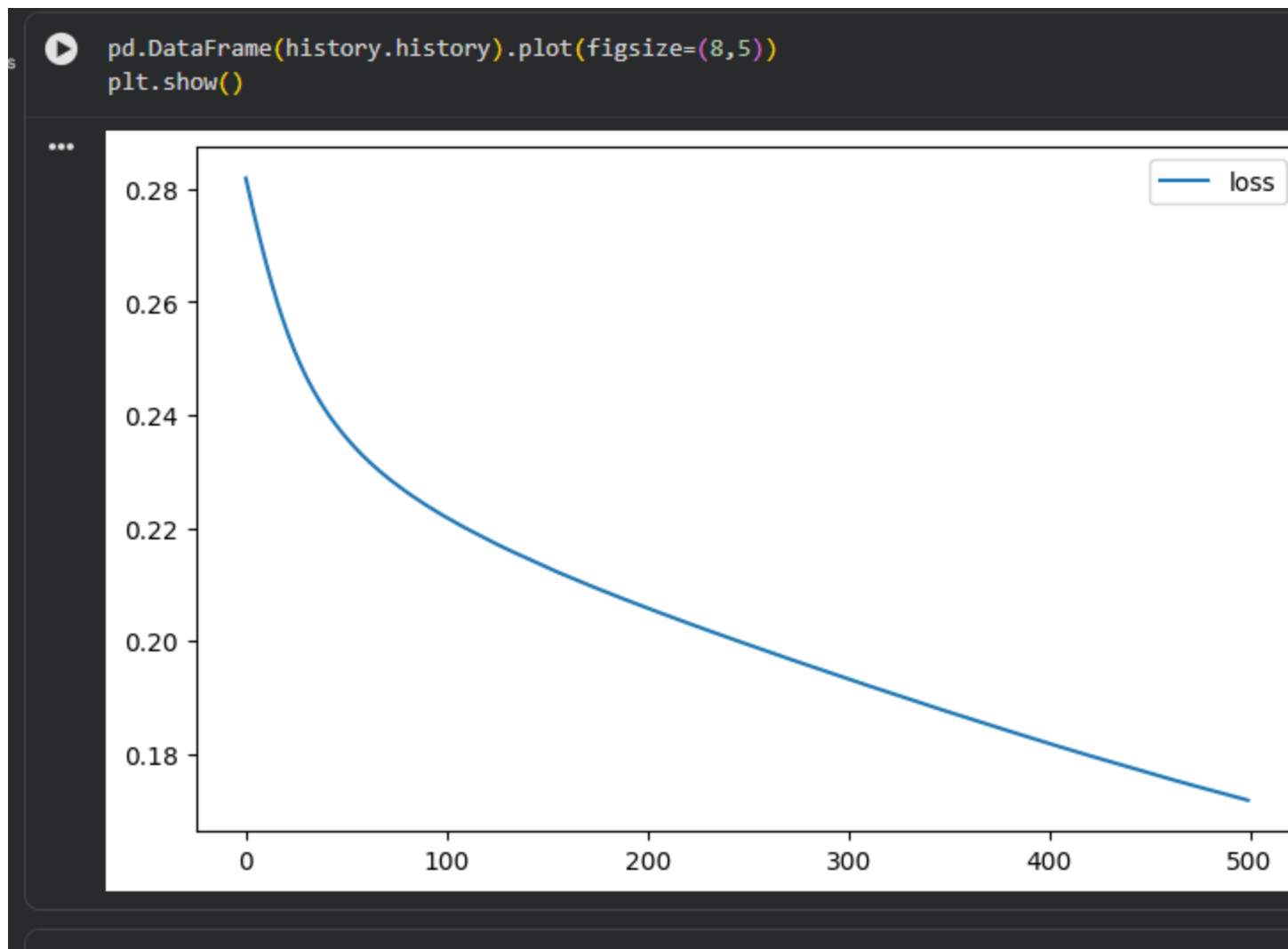
[5]



```
plt.plot(history.history['loss'])  
plt.show()
```

...





Capas extra:

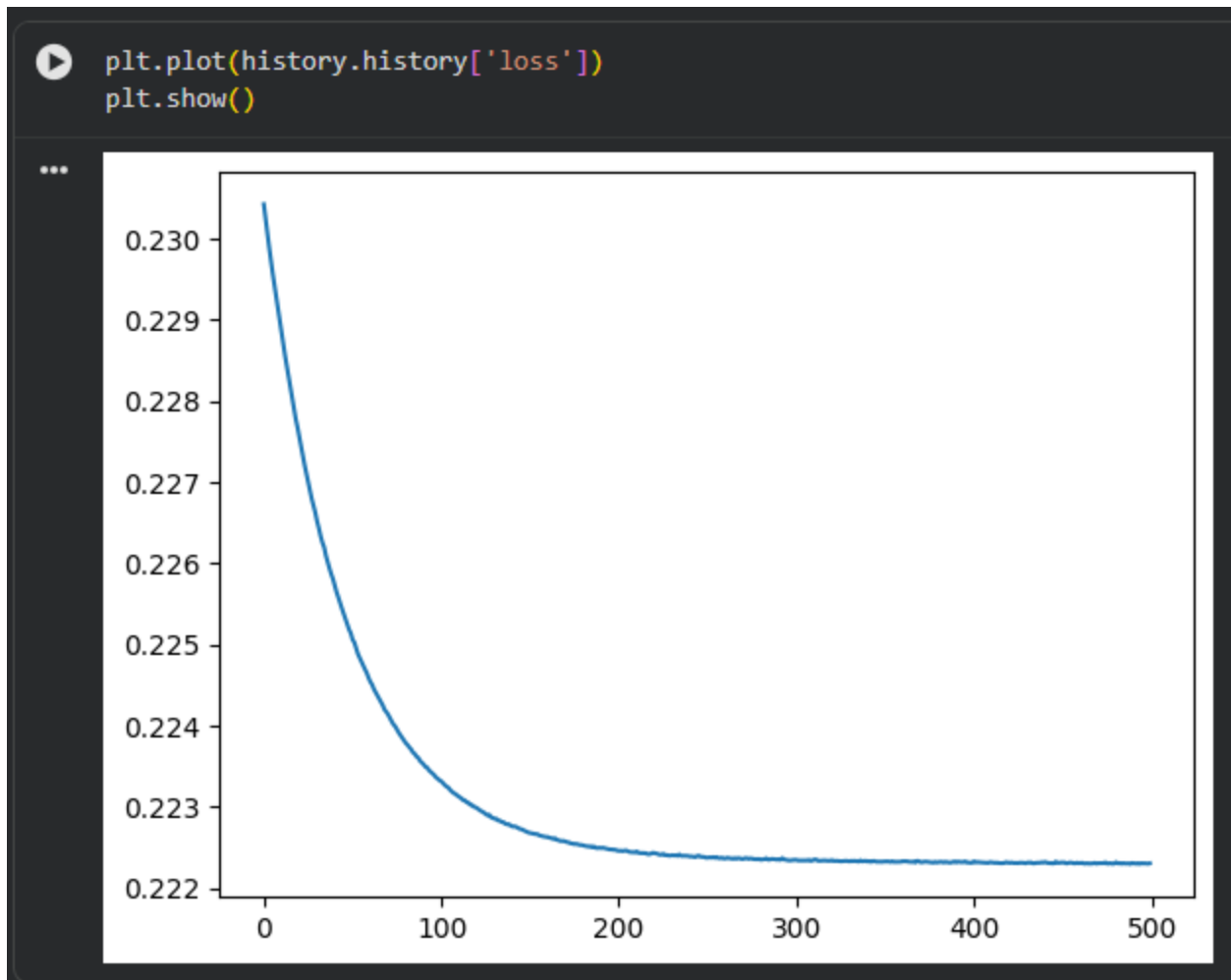
```
model = keras.Sequential([
    layers.Dense(3, activation="sigmoid", name="layer1", input_shape=(4,)),
    layers.Dense(3, activation="sigmoid", name="layer2"),
    layers.Dense(3, activation="sigmoid", name="layer3"),
    layers.Dense(3, activation="sigmoid", name="layer4"),
])

model.summary()
```

... /usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_3"

Layer (type)	Output Shape	Param #
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12
layer3 (Dense)	(None, 3)	12
layer4 (Dense)	(None, 3)	12

Total params: 51 (204.00 B)
Trainable params: 51 (204.00 B)
Non-trainable params: 0 (0.00 B)



Implementacion manual:

original

```

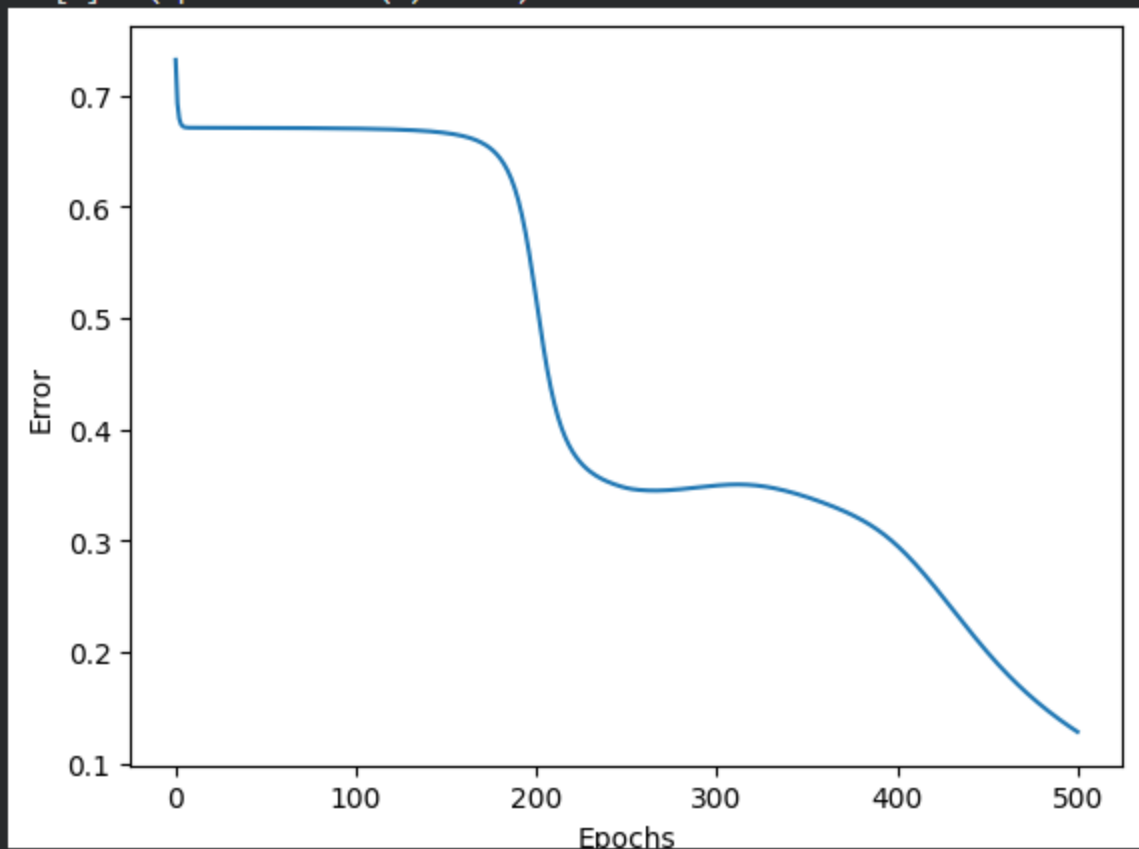
    for i in range(len(hidden_layer1)):
        hidden_layer1[i][0] += alpha * delta_hidden1[i] * 1 # Bias update
        for j in range(len(hidden_layer1[i])-1):
            hidden_layer1[i][j+1] += alpha * delta_hidden1[i] * X[d][j] # Input is X[d]

    E.append( calculate_error() )

Error_vec = np.asarray(E)
plt.plot(Error_vec)
plt.xlabel('Epochs')
plt.ylabel('Error')
plt.show()

```

... /tmp/ipykernel_910/4159385484.py:4: DeprecationWarning: Conversion of an array with
`w[i] = (np.random.rand(1) - 0.5)`



Capas extra:

[10]

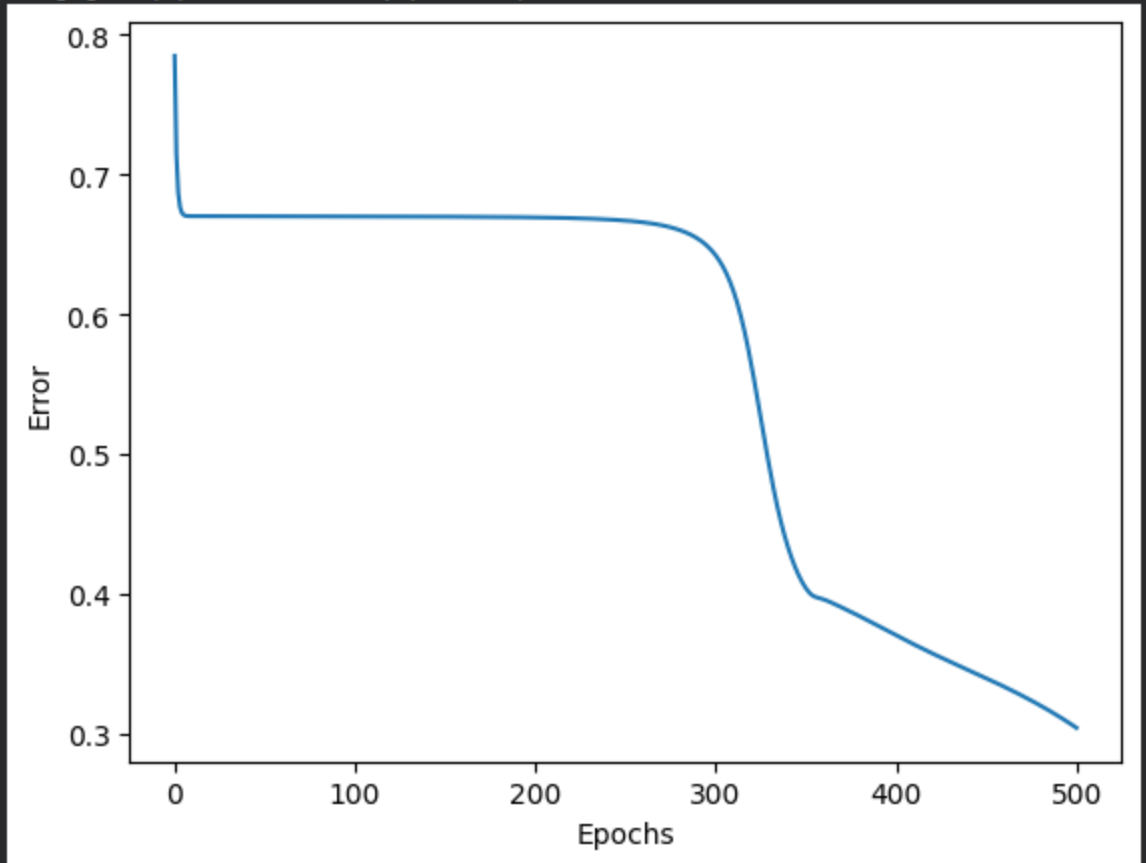
✓ 21s



```
plt.plot(Error_vec)
plt.xlabel('Epochs')
plt.ylabel('Error')
plt.show()
```



... /tmp/ipykernel_910/4159385484.py:4: DeprecationWarning: Conversion of an array w
w[i] = (np.random.rand(1) - 0.5)





🔍 Comandos + Código ▾ + Texto ▶ Ejecutar todo ▾



[1]



```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.utils import to_categorical
from sklearn import datasets
```

[2]



```
iris = datasets.load_iris()
X = iris.data
target = iris.target
Y = to_categorical(target).astype("uint8")
```

[3]



0 s



```
idx = 0
print(X[idx])
print(target[idx])
print(Y[idx])
idx = 50
print(X[idx])
print(target[idx])
print(Y[idx])
idx = 100
print(X[idx])
print(target[idx])
print(Y[idx])
```



```
... [5.1 3.5 1.4 0.2]
0
[1 0 0]
[7. 3.2 4.7 1.4]
1
[0 1 0]
[6.3 3.3 6. 2.5]
2
[0 0 1]
```





Comandos

+ Código ▾

+ Texto

▶ Ejecutar todo ▾



```
[ ] import tensorflow as tf
    from tensorflow import keras
    from tensorflow.keras import layers

    import numpy as np
    import pandas as pd
    import matplotlib.pyplot as plt
    from tensorflow.keras.utils import to_categorical
    from sklearn import datasets
```

```
[ ] iris = datasets.load_iris()
    X = iris.data
    target = iris.target
    Y = to_categorical(target).astype("uint8")
    # 0 => [1 0 0]
    # 1 => [0 1 0]
    # 2 => [0 0 1]
```

```
[ ] idx = 0
    print(X[idx])
    print(target[idx])
    print(Y[idx])
```

▼ [5.1 3.5 1.4 0.2]
0
[1 0 0]

```
[ ] ▶ model = keras.Sequential(
    [
        layers.Dense(3, activation="sigmoid", name="layer1", input_shape=(4,)),
        layers.Dense(3, activation="sigmoid", name="layer2"),
        layers.Dense(3, activation="sigmoid", name="layer3"),
        layers.Dense(3, activation="sigmoid", name="layer4"),
    ]
)

    model.summary()
```

▼ ... /usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106:
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_3"

